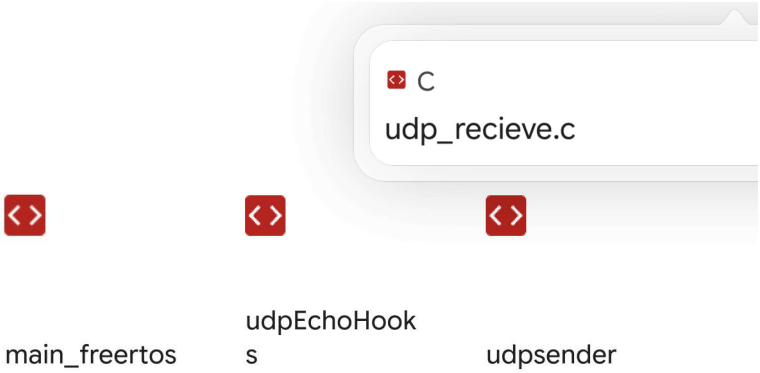


explain about this project in detail for beginner level understanding

This project is a **UDP Sender application** running on an embedded microcontroller using **FreeRTOS** and the **TI-NDK (Network Developer's Kit)** network stack. It sends a periodic "Ping" packet every second over an Ethernet cable directly to a target receiver board at IP address `192.168.1.102` on UDP port `1000`. main_freertos.c + 3

High-Level Architecture

```
+-------------------------------------------------------------------+
| 1. main_freertos.c                                                |
|    - Initializes hardware and UART display                        |
|    - Boots the TI-NDK network stack                               |
|    - Hands execution over to FreeRTOS Task Scheduler              |
+-----------------------------------+-------------------------------+
                                    |
                                    v
+-------------------------------------------------------------------+
| 2. udpEchoHooks.c (NDK Callback)                                  |
|    - Hardware acquires/configures IP                              |
|    - netIPAddrHook() fires automatically                          |
|    - Spawns the worker thread: udpSenderFxn                       |
+-----------------------------------+-------------------------------+
                                    |
                                    v
+-------------------------------------------------------------------+
| 3. udpsender.c (Application Logic)                                |
|    - Registers with NDK file descriptors (fdOpenSession)          |
|    - Opens a standard BSD UDP socket                              |
```

```
|    - Sends "Ping from Board 1 [Seq: N]" every 1 second via sendto |
+-------------------------------------------------------------------+
```

C
udp_recieve.c

Step-by-Step Breakdown of Each File

1. `main_freertos.c` (System Entry Point)

This is the starting file when the microcontroller boots. main_freertos.c

Hardware & Console Init: main_freertos.c

`Board_init()` initializes board-specific clocks, GPIOs, and power lines. main_freertos.c

`Display_init()` and `Display_open(Display_Type_UART, NULL)` set up a serial UART console so debug messages can print to your PC's terminal. main_freertos.c

Network Stack Boot: main_freertos.c

`ti_ndk_config_Global_startupFxn()` kicks off background network services and Ethernet drivers. main_freertos.c

Starting the OS: main_freertos.c

`vTaskStartScheduler()` passes control to FreeRTOS. The RTOS begins managing threads and timers. main_freertos.c

Stack Overflow Hook: main_freertos.c

`vApplicationStackOverflowHook(...)` catches any task whose memory usage exceeds its allotted stack, trapping it in an infinite loop instead of silently corrupting memory. main_freertos.c

2. `udpEchoHooks.c` (Network Event Listener)

This file connects network stack events to your application logic. udpEchoHooks.c

`netIPAddrHook(...)` : udpEchoHooks.c

This function is an **NDK hook callback**. It is automatically executed by the TI-NDK stack the moment the Ethernet interface comes up and acquires an IP address (whether assigned statically or via DHCP). udpEchoHooks.c

It logs whether the IP was added or removed and prints the assigned IP in standard `A.B.C.D` format. udpEchoHooks.c

It initializes SlNet (`ti_net_SlNet_initConfig()`), which provides standard network abstraction layers. udpEchoHooks.c

C
udp_recieve.c

Thread Creation: udpEchoHooks.c

When `fAdd` is true (network is up) and `createTask` is true (runs only once), it configures thread attributes:

- Detached state (`PTHREAD_CREATE_DETACHED`). udpEchoHooks.c
- Priority `1` . udpEchoHooks.c
- Stack size `4096` bytes. udpEchoHooks.c

`pthread_create(&thread, &attrs, udpSenderFxn, NULL)` launches `udpSenderFxn` as an active task. udpEchoHooks.c

`serviceReport(...)` : udpEchoHooks.c

Monitors services like DHCP or Telnet, printing their status codes and catching network faults. udpEchoHooks.c

3. `udpsender.c` (The Sender Core)

This file handles the network socket and continuous data transmission. udpsender.c

`fdOpenSession(TaskSelf())` : udpsender.c

In TI-NDK, a task must register its thread ID with the stack's File Descriptor (FD) manager before calling BSD socket functions. Skipping this call prevents sockets from working. udpsender.c

Opening UDP Socket: udpsender.c

`socket(AF_INET, SOCK_DGRAM, IPPROTO_UDP)` creates a connectionless UDP socket. udpsender.c

Configuring Destination Address: udpsender.c

`destAddr.sin_family = AF_INET` (IPv4). udpsender.c

`destAddr.sin_port = htons(TARGET_PORT)` converts port 1000 to network byte order (big-endian). udpsender.c

`destAddr.sin_addr.s_addr = htonl((192U << 24) | ...)` sets destination IP `192.168.1.102` in binary format. udpsender.c

Transmission Loop (`while (1)`): udpsender.c

Formats a message string: `"Ping from Board 1 [Seq: 0]"` , `"Ping from Board 1 [Seq: 1]"` , etc. udpsender.c

`sendto(...)` fires the UDP packet out across the physic udpsender.c

C udp_recieve.c

Prints either confirmation of sent bytes or a failure message (e.g., if the cable is disconnected) to the UART display. udpsender.c

`sleep(1)` suspends the task for 1 second, letting FreeRTOS run other background maintenance tasks without pegging the CPU. udpsender.c

Cleanup (`shutdown`): udpsender.c

If an error occurs, `close(sock)` closes the socket and `fdCloseSession(TaskSelf())` cleanly deregisters the thread from NDK. udpsender.c

Summary Checklist for Running

1. **Physical Setup**: An Ethernet cable connected between your board and the receiver machine (or connected via a switch).
2. **IP Configuration**: Ensure the sender board's interface has an IP in the same subnet (e.g., `192.168.1.101/24`) so it can talk directly to `192.168.1.102` . udpEchoHooks.c + 1
3. **Receiving End**: The device at `192.168.1.102` must have a UDP listener running on port `1000` (such as Packet Sender, Netcat, or an equivalent board running `udpEchoFxn`) to view the incoming packets. udpEchoHooks.c + 1

main_freertos udp_recieve udpEchoHooks

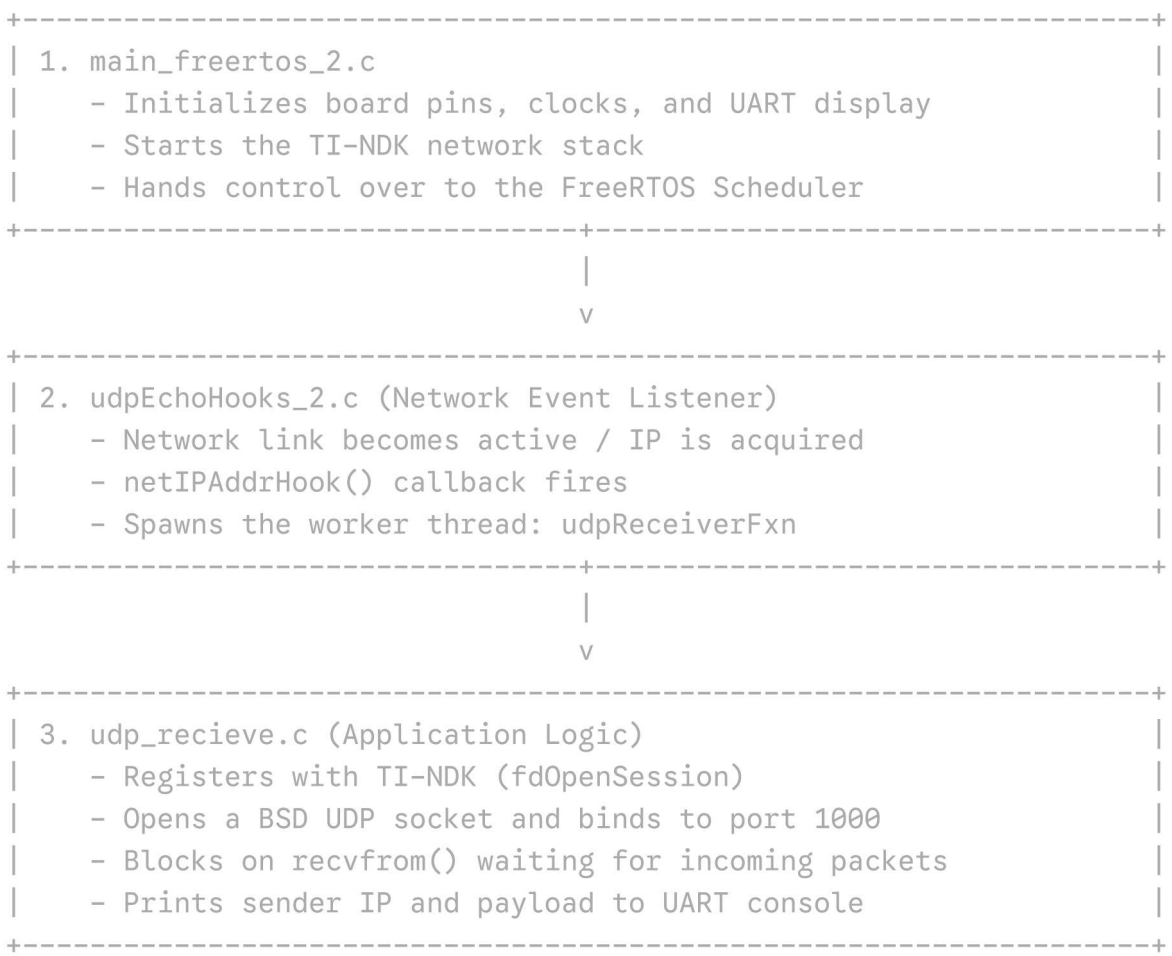
explain about this project in detail for beginner level understanding

C
udp_recieve.c

This project is the **UDP Receiver application** (Board 2). It acts as the re
to your sender project, running on a microcontroller with **FreeRTOS** ar
stack. udp_recieve.c + 3

Its primary job is to bind to UDP port `1000`, wait for incoming packets sent over Ethernet, extract the sender's IP and message, and print them out via the UART serial console.
udp_recieve.c

High-Level Architecture



File-by-File Breakdown

1. `main_freertos_2.c` (System Entry Point)

This file handles boot-up and hands CPU control over to the operating system. main_freertos.c

C
udp_recieve.c

Hardware & UART Init: main_freertos.c

`Board_init()` initializes device clocks, power domains, and low-level drivers. main_freertos.c

`Display_init()` and `Display_open(Display_Type_UART, NULL)` prepare the UART serial terminal so you can read logs on your PC. main_freertos.c

Network Stack Init: main_freertos.c

`ti_ndk_config_Global_startupFxn()` loads the TI-NDK stack in the background. main_freertos.c

Starting the OS: main_freertos.c

`vTaskStartScheduler()` starts FreeRTOS task scheduling. main_freertos.c

Stack Overflow Trap: main_freertos.c

`vApplicationStackOverflowHook(...)` traps the CPU in an infinite loop if any task overruns its stack buffer, preventing memory corruption. main_freertos.c

2. `udpEchoHooks_2.c` (Network Hook Callback)

This file manages network status changes and thread creation. udpEchoHooks.c

`netIPAddrHook(...)`: udpEchoHooks.c

A callback invoked automatically by TI-NDK when the Ethernet interface brings up or removes an IP address. udpEchoHooks.c

Prints whether the network was added or removed, along with the assigned IP address. udpEchoHooks.c

Calls `ti_net_SlNet_initConfig()` to initialize network socket layer services. udpEchoHooks.c

Spawning `udpReceiverFxn`: udpEchoHooks.c

When `fAdd` is true (link established) and `createTask` is true (runs only once):

Configures thread attributes: `PTHREAD_CREATE_DETACHED`, priority 1 and stack size `4096` bytes (`UDPHANDLERSTACK`).

Runs `pthread_create(&thread, &attrs, udpRec...` to start the receiver task. udpEchoHooks.c

C
udp_recieve.c

`serviceReport(...)` : udpEchoHooks.c

Monitors standard network services (such as DHCP or Telnet) and logs failures if network configuration encounters errors. udpEchoHooks.c

3. `udp_recieve.c` (The UDP Listener Core)

This file sets up the socket, binds it to listen locally, and reads packets. udp_recieve.c

`fdOpenSession(TaskSelf())` : udp_recieve.c

In TI-NDK, any task that uses BSD sockets must register itself with the file descriptor manager before calling socket APIs. udp_recieve.c

Creating and Binding the Socket: udp_recieve.c

`socket(AF_INET, SOCK_DGRAM, IPPROTO_UDP)` opens a connectionless UDP socket. udp_recieve.c

Fills `serverAddr` :

`serverAddr.sin_family = AF_INET` (IPv4). udp_recieve.c

`serverAddr.sin_port = htons(LISTEN_PORT)` (listens on port `1000` , converted to network byte order). udp_recieve.c

`serverAddr.sin_addr.s_addr = htonl(INADDR_ANY)` tells the socket to accept incoming packets on any local IP assigned to the board. udp_recieve.c

`bind(sock, (struct sockaddr *)&serverAddr, sizeof(serverAddr))` assigns the port to the socket. If another process is using this port or the socket is bad, it fails here. udp_recieve.c + 1

Receiving Packets (`while (1)`): udp_recieve.c

`recvfrom(sock, buffer, sizeof(buffer) - 1, 0, (struct sockaddr *)&clientAddr, &addrlen)` is a **blocking call**. The thread pauses and yields CPU execution to other tasks until a UDP packet hits port 1000. udp_recieve.c + 1

When a packet arrives: udp_recieve.c

`bytesRcvd` contains the number of bytes received.

`buffer[bytesRcvd] = '\0'` null-terminates the i
it can be safely printed as a C-string. udp_recieve.c

C
udp_recieve.c

`ntohl(clientAddr.sin_addr.s_addr)` extracts the sender's IP address. udp_recieve.c

`Display_printf(...)` displays the sender's IP address and the message payload to the console. udp_recieve.c

Shutdown / Cleanup: udp_recieve.c

If the socket fails, `close(sock)` releases the socket descriptor and `fdCloseSession(TaskSelf())` deregisters the thread from NDK. udp_recieve.c

How Sender and Receiver Fit Together

1. **Board 1 (Sender):** Has target IP set to `192.168.1.102` on port `1000` and fires a UDP packet every 1 second. udpsender.c
2. **Board 2 (Receiver):** Has its local IP set to `192.168.1.102`, binds to port `1000`, and sleeps in `recvfrom()`. udp_recieve.c
3. **Execution:** When the packet arrives across the Ethernet cable, Board 2 wakes up, reads the buffer, and outputs:

Plaintext

```
[Received from 192.168.1.101]: Ping from Board 1 [Seq: 0]
```